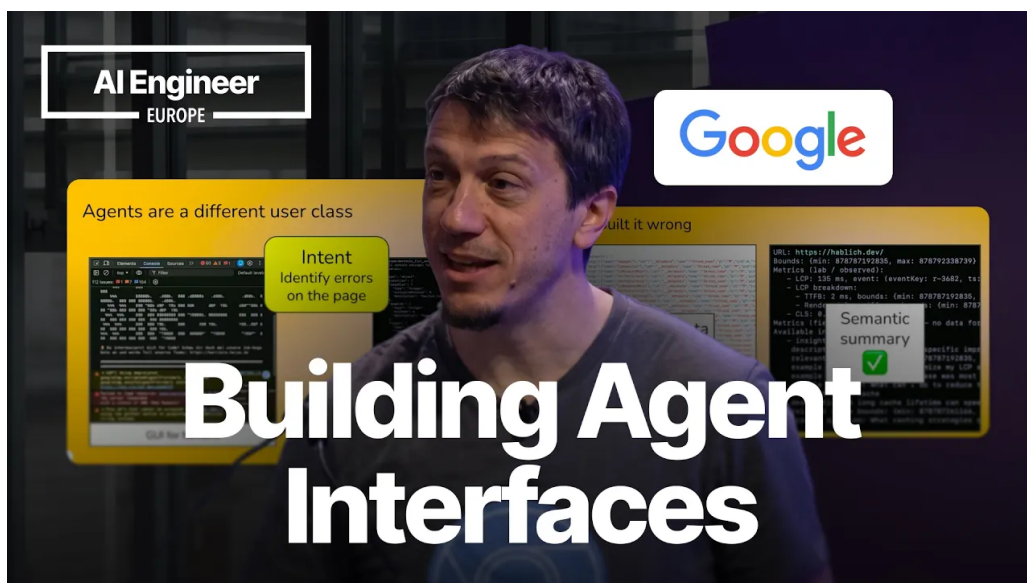




为 AI 代理设计接口

Chrome DevTools MCP 的工程实践

演讲者：Mikaël Hadik（Chrome DevTools 产品经理）

整理日期：2026 年 6 月 7 日

目录

1	引言：为 AI 代理构建开发者工具	2
1.1	演讲背景	2
1.2	Chrome DevTools 的双重身份	2
1.3	现场演示	3
2	核心问题：代理在盲目操作	3
2.1	一年半前的困境	3
2.2	数据过载的陷阱	4
2.3	语义摘要：化繁为简	5
3	用户群体差异：人类 vs 代理	5
3.1	截然不同的用户	5
3.2	代理偏好的文本界面	6
4	令牌消耗效率：界面的燃油效率	6
4.1	核心指标：每成功结果令牌数	6
4.2	前提条件：成功才是关键	7
4.3	降低消耗的三大策略	7
4.3.1	策略一：分层分类（Tiered Categorization）	7
4.3.2	策略二：精简模式（Compact Mode）	7
4.3.3	策略三：命令行界面（CLI Interface）	8
5	错误恢复机制：从失败中学习	8
5.1	错误的代价	8
5.2	第一层：有用的错误提示	9
5.3	第二层：主动绕行（Proactive Detours）	9
5.4	第三层：技能（Skills）与自助修复	9
6	工具可发现性：描述即界面	10
6.1	从单体工具到工具矩阵	10
6.2	MCP 工具描述的质量危机	10
6.3	优化的四个方向	11
6.4	技能（Skills）的双刃剑效应	11
7	信任边界：安全永不妥协	12
7.1	自动连接的双面性	12
7.2	三层信任模型	12

7.2.1	Tier 1: 本地开发环境	13
7.2.2	Tier 2: CI 环境	13
7.2.3	Tier 3: YOLO 模式	13
7.3	关键设计原则	14
8	总结与展望	14
8.1	用户体验的融合	14
8.2	四大核心要点	15
8.3	代理即用户细分	15

1 引言：为 AI 代理构建开发者工具

1.1 演讲背景

本次演讲由 Chrome DevTools 团队的产品经理 Mikaël Hadik 带来，主题是如何为 AI 代理（Agent）设计和构建接口。演讲者首先询问了现场观众的使用情况：

- 大多数人已经在使用 MCP 服务器或命令行工具
- 约一半人已经搭建并成功部署过 MCP 服务器
- 仅约 10% 的人尝试过 Chrome DevTools MCP



图 1: 演讲开场：询问现场观众对 MCP 和 Chrome DevTools Agent 的使用情况（00:20）

1.2 Chrome DevTools 的双重身份

Chrome DevTools（人类版）每天被数百万网页开发者用于调试网页，直接集成在 Chrome 浏览器中。开发者用它调试网页、查找错误、进行性能分析等。

而 **Chrome DevTools Agent** 则是专为 AI 代理设计的版本——同样强大的能力，但面向完全不同的用户群体。

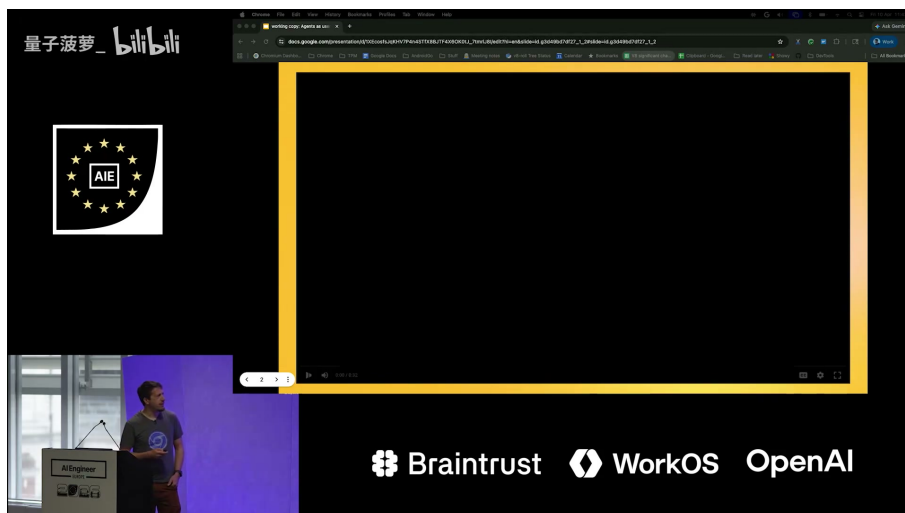


图 2: Chrome DevTools Agent 演示：左侧为 Gemini 界面输入提示，右侧配置 MCP 服务器并打开 Chrome 浏览器进行调试（01:28）

1.3 现场演示

演讲者展示了一个典型的工作流程：

1. 在 Gemini 界面中输入提示内容
2. 配置 MCP 服务器连接到 Chrome DevTools Agent
3. 代理自动打开 Chrome 浏览器并进行调试
4. 执行性能追踪（Performance Tracing）
5. 分析返回的追踪数据，进入性能分析界面
6. 采取优化行动，使网页更快
7. 验证优化后是否真的有性能提升

这个演示传达了一个核心信息：这套方案适用于任何 MCP 客户端和代理，无论是 Gemini、Claude Code 还是其他框架。

2 核心问题：代理在盲目操作

2.1 一年半前的困境

Chrome DevTools 团队之所以发布 DevTools Agent，是因为他们观察到编码代理存在严重的”盲目操作”问题：

”他们生成代码很出色，但无法验证实际操作是否正确。”

当时团队以为只要给模型输入足够多的数据就能解决问题。但事实并非如此。

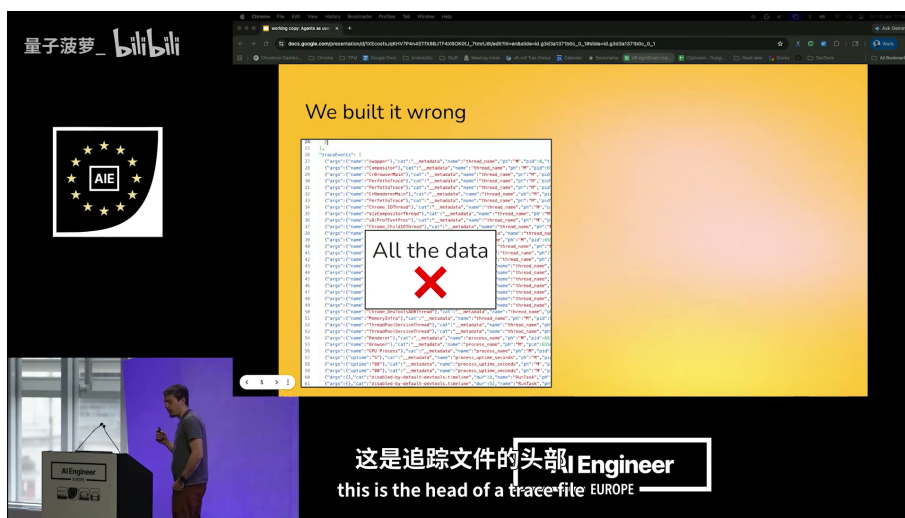


图 3: 追踪文件头部示例：包含数兆字节数据的性能配置文件，相当于约五十万行 JSON 数据（04:00）

2.2 数据过载的陷阱

一个性能追踪文件通常包含数兆字节的数据，相当于约 50 万行 JSON。一年半前，这些数据对模型进行推理来说实在太多了——远远超出了上下文窗口的容量。

这导致代理进入低效状态：面对海量原始数据，模型既无法有效理解，也无法做出正确的优化决策。

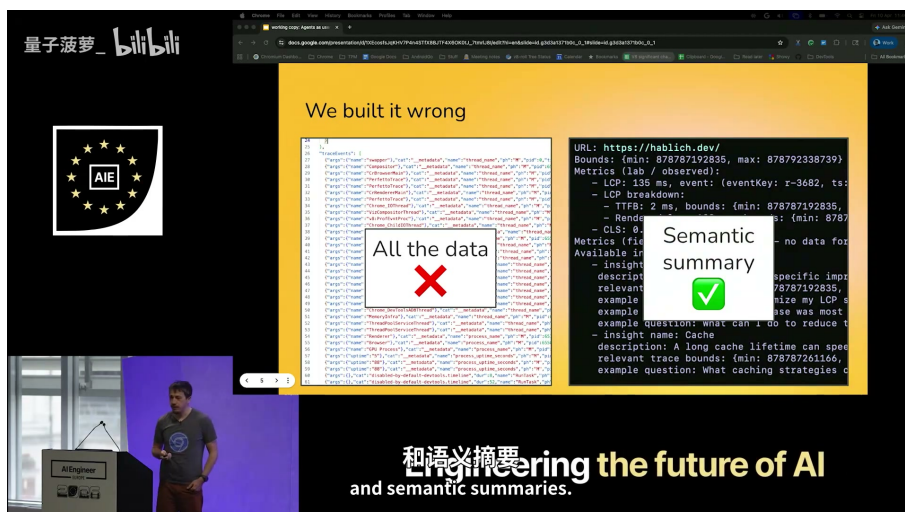


图 4: 语义摘要示例：将原始追踪数据转换为 Markdown 格式的关键性能指标摘要（04:50）

2.3 语义摘要：化繁为简

团队想到的解决方案是 **语义摘要 (Semantic Summary)**:

- 不再让代理阅读完整的原始追踪数据
- 而是返回结构化的 Markdown 格式摘要
- 直接指向关键信息，如最大内容渲染时间 (LCP) 等核心 Web 指标

这种摘要不仅可供代理阅读，也可供其他工具进行后处理。关键在于：**不需要强迫代理处理所有 Token。**

3 用户群体差异：人类 vs 代理

3.1 截然不同的用户

演讲者提出了一个“恍然大悟”的观点：**AI 代理是与人类完全不同的用户群体。**

虽然双方都希望识别页面错误并修复它们（共享意图与目标），但思考方式和认知瓶颈存在根本差异：

维度	人类开发者	AI 代理
信息偏好	视觉界面	非视觉界面
关键需求	布局设计、颜色辅助	模式清晰度、数据密度
错误识别	通过视觉扫描快速定位	通过结构化文本分析

表 1: 人类开发者与 AI 代理的界面需求对比



图 5: Chrome DevTools 控制台界面：对人类友好的视觉设计，包含颜色编码和布局层次 (06:00)

3.2 代理偏好的文本界面

代理显然更偏好 非视觉界面。它们重视：

- 模式清晰度（Pattern Clarity）：数据结构清晰、一致
- 数据密度（Data Density）：在单位内容中包含更多有用信息
- 可操作性：能够直接基于文本做出决策

右侧的性能端点返回的 Markdown 摘要就是一个很好的例子——代理非常喜欢这种纯文本、非视觉的界面。

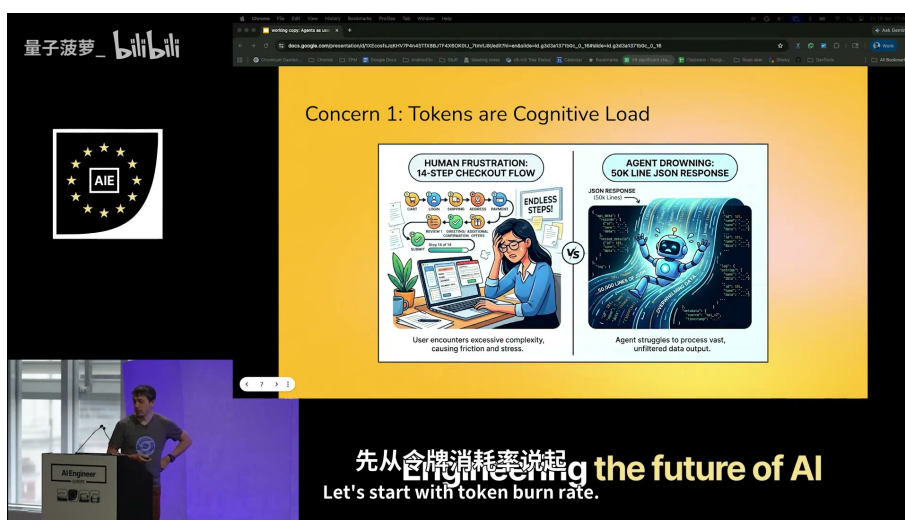


图 6: 性能端点返回的文本模式：代理偏好的高密度、结构化数据界面（07:13）

4 令牌消耗效率：界面的燃油效率

4.1 核心指标：每成功结果令牌数

演讲者引入了关键指标 **Tokens per Successful Outcome**（每成功结果令牌数），用来衡量接口的“燃油效率”。

”它衡量的是：在完成一次完整用户旅程的前提下，消耗了多少令牌。”

这个指标由两个维度构成：

1. **效果（Effectiveness）**：代理是否完成了整个用户旅程？功能意图是否真正实现？——是或否
2. **效率（Efficiency）**：完成该任务消耗了多少令牌？包括两次调用的令牌成本、最终时长等

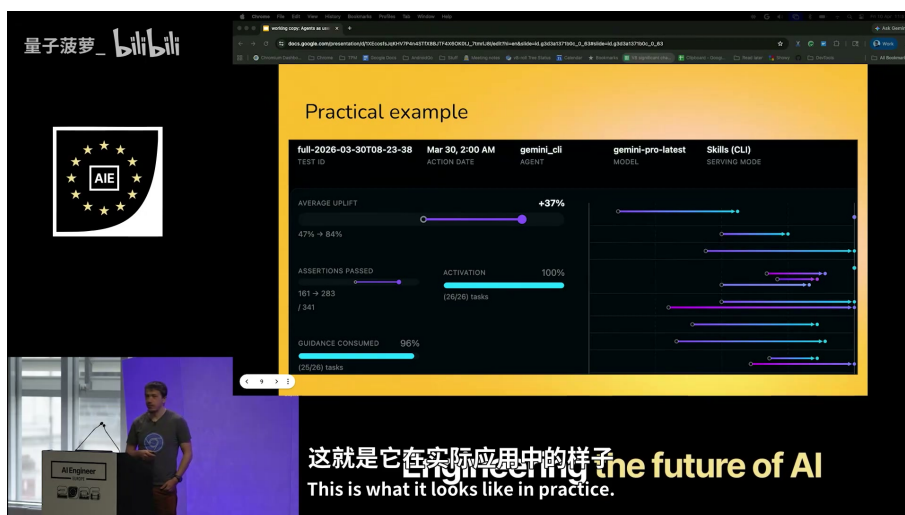


图 7: 项目仪表盘: 展示不同使用场景下的每成功结果令牌数, 条形越长代表工具越有效 (10:05)

4.2 前提条件: 成功才是关键

这个指标有一个重要前提: 如果无法到达目的地, 燃油效率毫无意义。

因此指标命名为”每成功结果令牌数”而非”每结果令牌数”。务必同时测量:

- 效果 (是否成功)
- 效率 (消耗多少 Token)

而且, 这个指标 **无法全局测量**——在不同用户旅程和任务类别间差异极大。应该在同类用户旅程内进行比较。

4.3 降低消耗的三大策略

4.3.1 策略一: 分层分类 (Tiered Categorization)

通过命令行参数隐藏专业工具。例如:

- Chrome 扩展调试工具——并非所有人都在开发扩展
- 不需要将其加入默认右键菜单

4.3.2 策略二: 精简模式 (Compact Mode)

将分类推向极限, 仅展示三个核心工具:

1. 静态页面工具
2. 导航页面工具

3. 脚本工具

这对上下文窗口非常有利，但存在权衡：暴露的工具越少，代理可能需要更多步骤才能达成目标。

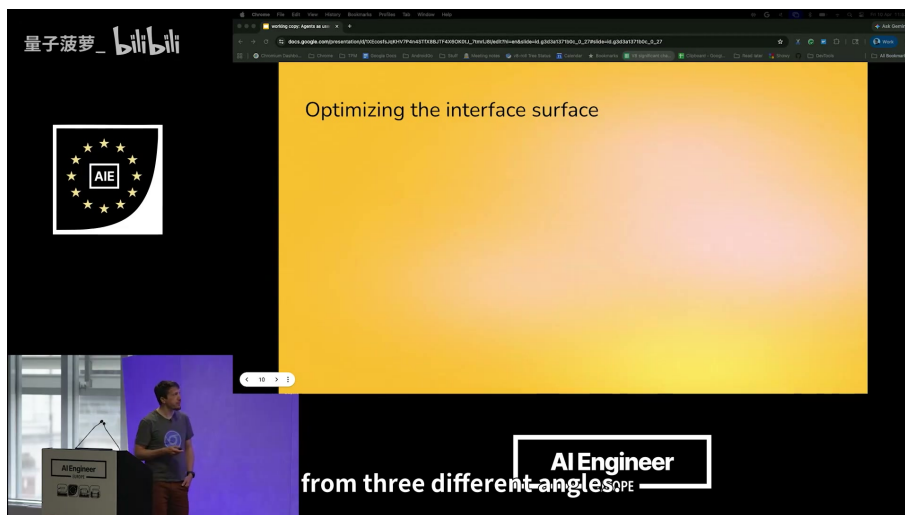


图 8: 精简模式示意图：通过减少工具暴露数量来降低上下文窗口压力（11:15）

4.3.3 策略三：命令行界面（CLI Interface）

除了 MCP 服务器，团队还提供了命令行界面，支持代理链式执行命令进行后处理。例如：

- 通过 CLI 命令提取可访问性树（Accessibility Tree）
- 将结果传递到点击命令
- 处理过程在本地电脑上完成，模型无需处理所有 Token

5 错误恢复机制：从失败中学习

5.1 错误的代价

每次代理遇到错误时，都会消耗额外的 Token——因为它需要重试、理解问题并处理相关情况。如果代理卡住，效率再高也毫无意义。

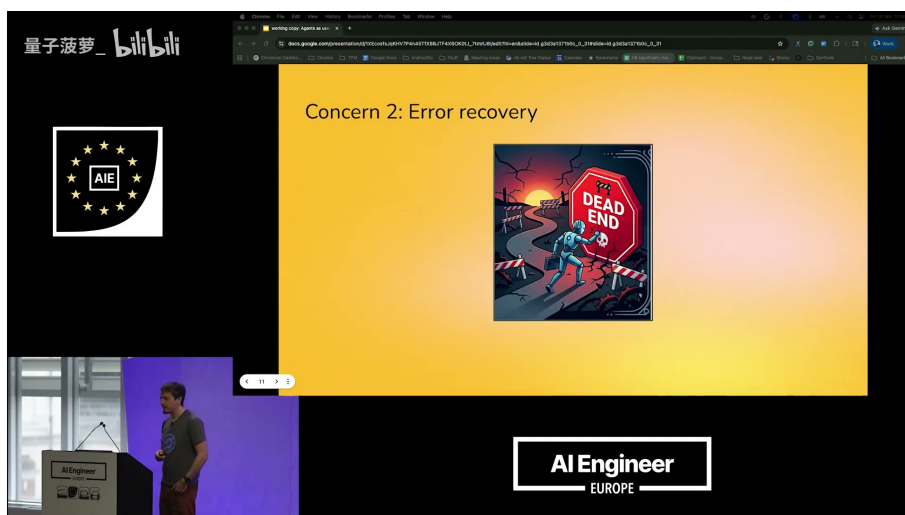


图 9: 错误恢复连续体：从基础错误提示到主动绕行和自助修复的多层策略（13:30）

5.2 第一层：有用的错误提示

看似显而易见，但实际上并非如此。团队对所有工具进行了多次迭代优化错误提示。
示例：

旧提示：“无法在选定页面返回”

优化后：“特定工具历史记录中导航未找到。请尝试使用浏览器的回退功能。”

添加最后一句让代理能够感知问题并自主解决，无需人工干预。

5.3 第二层：主动绕行（Proactive Detours）

每个代理底层都有一个模型，模型基于特定数据训练。有时需要调用训练数据之外的策略。

团队为性能分析引导代理进行绕行：

- 不直接让代理运行最轻量的性能追踪
- 而是提供诊断操作手册（Diagnostic Playbook）
- 让代理按步骤执行更稳健的分析流程

5.4 第三层：技能（Skills）与自助修复

团队引入了名为 **Troubleshooting**（故障排除）的技能：

- 当很多人在设置 Chrome DevTools MCP 服务器时遇到问题

- 故障排除技能自动启动
- 帮助人类和代理修复设置问题
- 启用代理的自我反馈机制

这提升了产品的韧性，增强了代理框架的稳定性，帮助避免重复错误。

6 工具可发现性：描述即界面

6.1 从单体工具到工具矩阵

最初的设计只有一个名为”调试网页”的单体工具。其他代理发送提示到该工具：

”嘿，调试这个网页，这里存在一些问题。”

但从工程角度看，这并不奏效。团队将其拆分为 **25 个不同工具**——但这并未解决问题，只是将问题转移了：

”现在代理拥有 25 种可用工具，如何确定何时使用哪个工具呢？”



图 10: 工具描述优化示例：清晰解释工具核心功能并提供使用指南（15:28）

6.2 MCP 工具描述的质量危机

根据相关研究，**97% 的 MCP 工具描述存在质量缺陷**。这非常重要，因为：

”模式（Schema）是代理的 UI。”

优化 UI 需要权衡取舍：

- 描述越详细 → 上下文容量占用越大
- 描述越简短 → 代理理解偏差越大
- 描述过多 → 导致代理错误使用工具

这是一个空间权衡（Space Trade-off），需要找到最小可行描述（Minimum Viable Description）。



图 11: 性能追踪工具描述示例：包含明确的激活条件和使用场景（16:15）

6.3 优化的四个方向

1. 找到目标（Find the Target）：清晰说明工具解决什么问题
2. 命名清晰：工具名称应准确反映功能
3. 激活条件明确：说明何时应该调用该工具
4. 关联上下文：帮助代理建立工具与任务之间的连接

示例：性能追踪工具的描述明确提到”从性能问题和核心 Web LCP 指标中提取数据”，代理就能建立连接：”哦，如果要优化页面加载性能，我应该使用这个工具。”

6.4 技能（Skills）的双刃剑效应

技能可以大幅提升工具的可发现性和使用效果，但也存在权衡：

- 堆叠太多技能 → 问题转移，再次遇到相同困境
- 代理可能在不该调用时调用技能

- 内容大小会增加，Token 消耗上升

团队认为这像是”永无止境的最小可行描述追求”——因为模型和框架也在持续更新。

7 信任边界：安全永不妥协

7.1 自动连接的双面性

Chrome DevTools Agent 提供了自动连接功能。对人类用户来说，使用编码代理时共享屏幕就像说：

”我卡在这里了，请帮我调试这个错误，并修复我看到的问题。”

这是一个很神奇的功能。但用户很快提出功能请求：

- ”为什么我每次都要点击允许？”
- ”请记住我的选择。”

在传统 UX 设计中，减少摩擦是明确胜利。但在将工作委托给代理的世界里，摩擦实际上是通过标识（Friction is Identity）——因为它是信任边界的一部分。

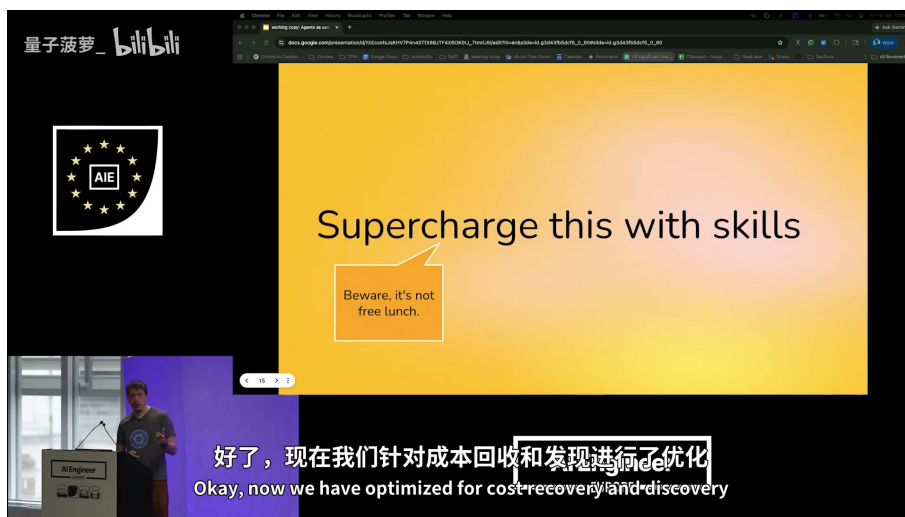


图 12: 信任边界三层模型：浏览代理、本地开发环境、CI 环境的差异化权限设计（18:35）

7.2 三层信任模型

团队设计了 三层信任模型：

层级	环境	安全策略
Tier 1	本地开发环境	人 类 在 循 环 （Human-in-the-Loop），每次授权，使用默认 Chrome 配置文件
Tier 2	CI 环境	受控环境，数据隔离，使用容器化技术和独立 Chrome 配置文件
Tier 3	YOLO 模式	完全互联网访问，需域名限制和提示注入防护

表 2: Chrome DevTools Agent 信任边界三层模型

7.2.1 Tier 1: 本地开发环境

- 人类在循环
- 人类希望授予默认 Chrome 配置文件的访问权限
- 对可访问数据以时间绑定的方式授予代理权限

7.2.2 Tier 2: CI 环境

- 运行在持续集成环境中的代理
- 受控环境，但已被隔离
- 应使用数据隔离（容器化技术、独立 Chrome 配置文件）
- 提供远程调试端口机制

7.2.3 Tier 3: YOLO 模式

- 具有完全互联网访问权限的代理
- 本质危险：任何网页都能向代理注入提示文本（Prompt Injection）
- 必须像第二层那样操作
- 确保域名限制和提示注入防护措施

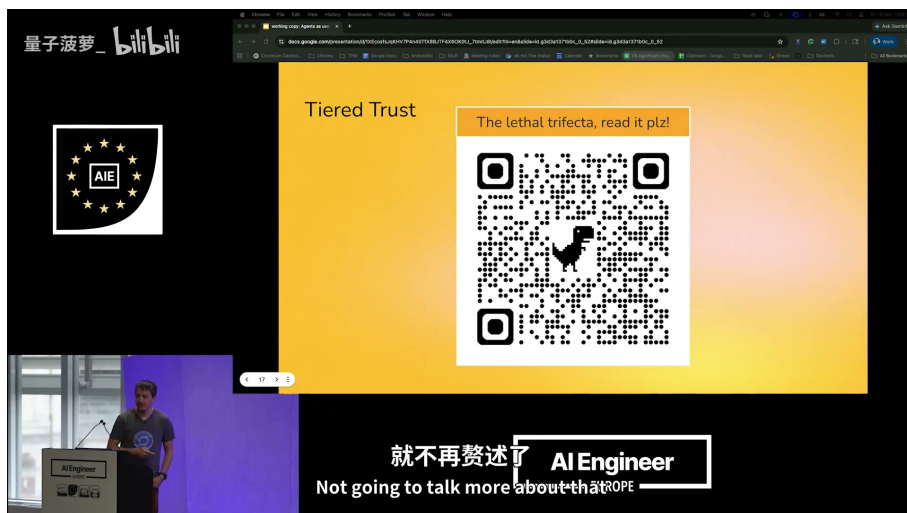


图 13: 远程调试端口与隔离机制：为不同安全层级提供差异化连接方案（19:50）

7.3 关键设计原则

- 人类每次都需要授权（针对 Tier 1）
- 不同层级的代理不应共享安全模型信息
- 可以共享 Chrome DevTools 等工具，但安全上下文必须隔离

8 总结与展望

8.1 用户体验的融合



图 14: 演讲总结：四个核心要点与下一代用户的展望（21:30）

8.2 四大核心要点

演讲者总结了 Chrome DevTools Agent 的四个核心设计原则：

1. 衡量燃料效率

使用 **每成功结果令牌数 (Tokens per Successful Outcome)** 衡量接口的燃油效率。在同类用户旅程内比较，持续优化。

2. 将错误转化为恢复机制

提供有用的错误提示、主动绕行策略、操作手册和技能支持。让代理能够自我修复，提升系统韧性。

3. 意图的外部描述

优化工具描述，帮助代理建立工具与任务之间的连接。在描述质量和上下文占用之间找到平衡。

4. 永不妥协的信任

为便捷代理保持信任边界。区分三层安全模型，确保人类始终掌握关键授权决策。

8.3 代理即用户细分

”代理只是另一种用户细分类型。”

用户体验正在融合传统体验与代理体验。除了功能需求，还需满足非功能性需求：效率、可发现性、安全性、稳定性等。

”让我们帮助我们的下一位用户——帮助我们帮助他们。”

——Mikaël Hadik